PRD: AI Data Science Agent

Author: Kavish | **Status:** Draft v0.1 | **Type:** Portfolio / MVP project

1. Overview

The AI Data Science Agent automates the repetitive first mile of a data science workflow. A user uploads a CSV and the agent profiles it, proposes a cleaning plan, executes that plan, generates charts, suggests and benchmarks baseline ML models, and produces a single summary report.

The core design principle: **the LLM plans and narrates; all data work is deterministic.** The LLM never touches raw data or writes free-form code. It returns structured plans that are validated and then executed by pandas and scikit-learn.

2. Problem Statement

Data scientists and students spend a large share of their time on repetitive work: checking types, nulls, duplicates and outliers, deciding how to clean them, making the same exploratory charts, and picking a baseline model. This work is formulaic but still needs judgment, which makes it a good fit for an LLM-assisted, human-approved workflow.

3. Goals and Non-Goals

Goals

- Go from CSV upload to a complete report in a few minutes with minimal manual effort.
- Keep every transformation reproducible, auditable and reversible.
- Keep the user in control through an approve/edit step before any data is changed.
- Give honest, measured model recommendations based on real baseline results.

Non-Goals (MVP)

- Not an AutoML system that tunes and deploys production models.
- No support for databases, streaming data or files beyond CSV.
- No deep learning, NLP or image models.

- No multi-user accounts or collaboration in v1.

4. Target Users

- **Primary:** students and junior data scientists who want a fast, guided starting point.
- **Secondary:** analysts and developers who need a quick look at an unfamiliar dataset.

5. User Flow

1. **Upload** a CSV (with a size and row limit).
2. **Profile:** the system scans column types, nulls, duplicates, outliers, cardinality and basic stats.
3. **Plan:** the LLM receives the profile (not raw rows) and returns a structured cleaning plan with a reason for each step.
4. **Review:** the user sees the plan, toggles or edits steps, and approves.
5. **Execute:** pandas applies the approved steps; a before/after log is recorded.
6. **Visualize:** charts are generated automatically (distributions, correlations, missingness, category counts).
7. **Model:** the user confirms a target column; the system detects the problem type, trains 2 to 3 baseline models and compares cross-validated scores.
8. **Report:** the user downloads two files: the cleaned dataset (CSV) and a complete process report. The LLM narrates findings; the system assembles both.

6. Functional Requirements

6.1 Upload and Ingestion

- Accept CSV files with encoding detection and delimiter sniffing.
- Enforce a max file size and row count; show a clear error if exceeded.
- Generate a session ID for each upload.

6.2 Data Profiler (deterministic, no LLM)

- Infer per-column type: numeric, categorical, datetime, boolean, text, ID-like.
- Report null counts and percentages, duplicate row count, unique counts and top values.

- Detect outliers using IQR (default) with the method recorded.
- Output a compact JSON profile suitable for the LLM prompt.

6.3 Cleaning Planner (LLM)

- Input: the profile JSON only. No raw rows are sent by default.
- Output: JSON validated with Pydantic against a whitelist of operations, for example `drop_duplicates`, `drop_column`, `impute` (mean, median, mode, constant), `clip_outliers`, `cast_type`, `parse_datetime`, `standardize_text`.
- Each step includes the target column, parameters and a short rationale.
- Invalid or unknown operations are rejected and the LLM is re-prompted (bounded retries).

6.4 Plan Review UI

- Display steps as a list with rationale; allow enable/disable and parameter edits.
- Require explicit approval before execution.

6.5 Cleaning Executor (deterministic)

- Apply approved steps in order using pandas.
- Log row/column counts and null/outlier changes per step.
- Keep the original data untouched; export the cleaned dataset as a CSV file (a required deliverable) that keeps the original column names and order unless the user approved a rename or drop.

6.6 Chart Generation

- Auto-generate charts with matplotlib: histograms, box plots, correlation heatmap, missing-value map, top categories.
- Cap the number of charts to keep the report readable.

6.7 ML Suggestion and Baselines

- User selects (or confirms a suggested) target column.
- Problem type (classification or regression) is determined by rules on target dtype and cardinality.
- Train 2 to 3 baseline models with scikit-learn (for example logistic/linear regression and random forest), using proper train/test splits and cross-validation.

- Fit imputers, encoders and scalers on training data only to avoid leakage.
- LLM explains the results and the recommended model in plain language.

6.8 Report

- A complete process report (a required deliverable) covering, in order: dataset snapshot, issues found, cleaning plan and actions with approval status, before/after comparison, charts, ML analysis and narrated summary.
- Export as HTML (MVP) and PDF (stretch).
- Include a reproducible Python script of the executed steps (stretch).

6.9 Final Deliverables

After processing, the results page offers two required downloads:

1. **Cleaned dataset:** `cleaned_<original_name>.csv`, containing the data after all approved cleaning steps. The original upload is never modified.
2. **Process report:** `report.html` (PDF as a stretch), documenting the whole run: dataset snapshot before and after, issues found, each cleaning step with its reason, parameters, approval status and before/after counts, a before vs after comparison, charts, the ML analysis, and the narrated summary.

Optional extras that do not change the core flow: a single .zip bundle of both files, the reproducible Python script, and the chart PNGs.

7. Non-Functional Requirements

- **Determinism:** the same data and approved plan always produce the same output.
- **Safety:** no arbitrary code execution from LLM output; strict schema validation.
- **Privacy:** only aggregate profile data goes to the LLM; uploaded files are deleted after a retention window.
- **Performance:** profile and cleaning complete in seconds for files up to the size limit; the full pipeline in a few minutes.
- **Reliability:** graceful handling of malformed CSVs, empty columns and LLM failures.
- **Observability:** structured logs for each pipeline stage and LLM call.

8. Technical Architecture

- **Frontend:** Next.js and Tailwind CSS (upload, plan review, charts, report view).
- **Backend:** FastAPI with pandas, scikit-learn and matplotlib.
- **LLM:** OpenAI API for planning (structured JSON output) and narration.
- **Storage:** local or object storage for uploads and artifacts; session metadata in SQLite or Postgres.
- **Job handling:** background tasks (FastAPI BackgroundTasks or a queue) with progress polling.

Suggested API Endpoints

- `POST /sessions` upload a CSV and start profiling
- `GET /sessions/{id}/profile` fetch the profile
- `POST /sessions/{id}/plan` generate the cleaning plan
- `POST /sessions/{id}/execute` run the approved plan
- `POST /sessions/{id}/model` run baselines for a chosen target
- `GET /sessions/{id}/report` fetch or download the report

9. Success Metrics

- **Plan quality:** percentage of generated plans judged sensible on a set of test datasets (for example Titanic, Housing, Telco Churn).
- **Schema validity rate:** share of LLM responses that pass validation on the first attempt.
- **Time to report:** median time from upload to finished report.
- **Approval rate:** share of plan steps accepted without edits.
- **Reproducibility:** identical outputs across repeated runs of the same approved plan.

10. Risks and Mitigations

- **LLM proposes a harmful or wrong step:** whitelist, schema validation, human approval, audit log.
- **Data leakage in modeling:** fit all preprocessing on the training split only.
- **Messy or huge files:** size limits, encoding detection, chunked reading as a stretch.

- **Over-trusting the narration:** narration is generated only from computed numbers passed into the prompt.
- **Sensitive data:** profile-only prompts, retention limits, clear user notice.

11. Milestones

1. **M1:** Profiler and CSV ingestion
2. **M2:** Whitelisted cleaning executor with logging
3. **M3:** LLM planner with validated JSON output
4. **M4:** Chart generation
5. **M5:** Baseline model training and comparison
6. **M6:** Report generation and narration
7. **M7:** Next.js UI including plan approval flow
8. **M8:** Evaluation on public datasets, polish and demo

12. Stretch Features

- Follow-up chat ("why did you drop this column?", "redo with mean imputation")
- Export the pipeline as a Python script or notebook
- PDF report export
- Support for Excel and Parquet files
- Undo and redo of individual cleaning steps

13. Open Questions

1. **Target column:** should the user always choose it, or should the agent suggest candidates?
2. **Scope of models:** tabular classification and regression only, or also clustering and time series?
3. **Data privacy:** is sending the aggregate profile plus a few sample values to OpenAI acceptable, or profile statistics only?
4. **Deployment:** local demo only, or a hosted deployment (which affects auth, storage and cost)?
5. **Report format:** is HTML enough for the MVP, or is PDF required from day one?

6. **Audience:** is this mainly a portfolio piece, or a product you want real users for?